EXAM PREPARATION NOTES

Database Design & Normalization

ER-to-Relational Mapping, Design Guidelines, Functional Dependencies, and Normal Forms

Complete Revision Guide

With Diagrams, Algorithms & Practice Q&A

Table of Contents

Part 5: Quick Reference Cheat Sheet

Part 1: ER-to-Relational Mapping

These notes cover the comprehensive algorithm used to map a **Conceptual Schema (ER Model)** into a **Logical Schema (Relational Model)**.

> **Assumptions & Preliminaries**
>
> - The mapping creates tables with **simple, single-valued attributes**.
> - Relational model constraints (**primary keys, unique keys, referential integrity**) are defined during mapping.
> - Examples are based on the **COMPANY Database** (ER Schema and final mapped Relational Schema).

Step 1: Mapping of Regular (Strong) Entity Types

Algorithm

1. For each regular entity type *E*, create a relation (table) *R*.
2. Include all the simple attributes of *E* in *R*.
3. For composite attributes, include only their **simple component attributes**.
4. Choose one key attribute of *E* as the **Primary Key (PK)** for *R*. If the key is composite, the set of simple attributes forming it together become the PK.

> **Important Detail**
>
> If multiple keys were identified during conceptual design, keep this knowledge. It is used to specify additional (unique) keys and for indexing/analysis purposes later.

Relations created in this step are called **entity relations** because each tuple represents an entity instance. **Foreign keys and relationship attributes are NOT included yet** — they are added in later steps.

Example from COMPANY Database

Table 1.1 Entity Relations from COMPANY Database

Entity Type	Mapped Relation	Primary Key	Unique Key
EMPLOYEE	EMPLOYEE(Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary)	Ssn	—
DEPARTMENT	DEPARTMENT(Dname, Dnumber)	Dnumber	Dname
PROJECT	PROJECT(Pname, Pnumber, Plocation)	Pnumber	Pname

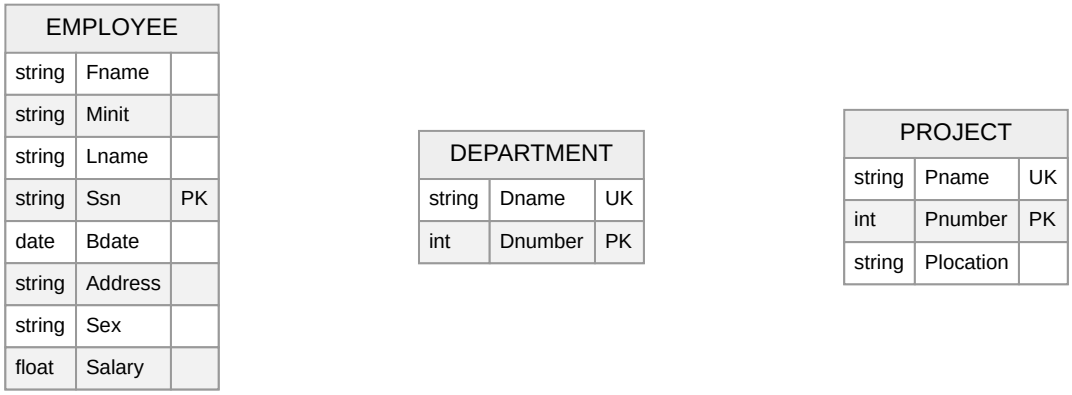

Figure 1.1: Entity Relations after Step 1 (Strong Entities)

Step 2: Mapping of Weak Entity Types

Algorithm

1. For each weak entity type *W* with owner entity *E*, create a relation *R*.
2. Include all simple attributes (or simple components of composite attributes) of *W*.
3. **Foreign Key:** Include the primary key attribute(s) of the owner entity *E* as foreign keys in *R*. This maps the identifying relationship.
4. **Primary Key:** The PK of *R* is the combination of the owner's PK and the partial key of the weak entity (if any).

> **Referential Triggers (CASCADE)**
>
> Use the **propagate (CASCADE)** option for the referential triggered action (ON UPDATE and ON DELETE) on the foreign key. A weak entity has an **existence dependency** on its owner.

> **Note on Nested Weak Entities**
>
> If a weak entity *W1* is owned by another weak entity *W2*, map *W2* first to determine its primary key before mapping *W1*.

Example from COMPANY Database

Weak Entity: DEPENDENT (Owner: EMPLOYEE)

Mapped Relation: DEPENDENT(Essn, Dependent_name, Sex, Bdate, Relationship)

- **FK:** Essn (references EMPLOYEE.Ssn)
- **PK:** {Essn, Dependent_name} — Dependent_name is the partial key

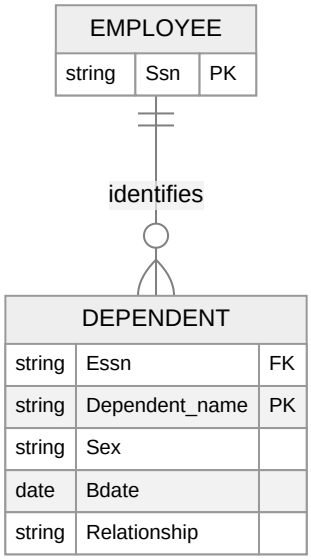

Figure 1.2: Weak Entity Mapping (DEPENDENT)

Step 3: Mapping of Binary 1:1 Relationship Types

For a 1:1 relationship *R* between entities *E1* and *E2*, there are **three approaches**:

Table 1.2 Three Approaches for 1:1 Relationship Mapping

Approach	Description	When to Use
1. Foreign Key (Preferred)	Choose one relation (entity with **total participation**) and include the PK of the other as a FK. Include any relationship attributes.	Most useful; minimizes NULLs
2. Merged Relation	Merge both entity types and the relationship into a single table.	Only when **both participations are total** (same number of tuples)
3. Cross-Reference	Create a third relation with PKs of both as FKs; one FK is unique.	Requires extra JOINs; rarely preferred

> **Exam Tip: Why Total Participation Gets the FK**
>
> If we put the FK in a table where participation is **partial** (e.g., putting Department_managed in EMPLOYEE), it creates massive NULL values. If only 2% of employees manage a department, 98% of tuples would have NULLs in that column.

Example from COMPANY Database

Relationship: MANAGES (1:1 between EMPLOYEE and DEPARTMENT)

- DEPARTMENT has **total participation** (every department MUST have a manager).
- Solution: Add **Mgr_ssn** (FK referencing EMPLOYEE.Ssn) and relationship attribute **Mgr_start_date** into the DEPARTMENT table.

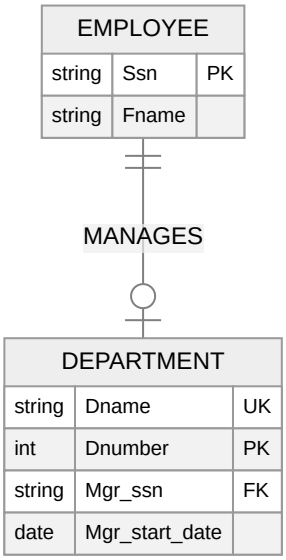

Figure 1.3: 1:1 Relationship Mapping (Foreign Key Approach)

Step 4: Mapping of Binary 1:N Relationship Types

Two possible approaches:

Table 1.3 Approaches for 1:N Relationship Mapping

Approach	Description	When to Use
1. Foreign Key (Preferred)	Identify the relation on the **N-side**. Include the PK of the 1-side as a FK. Include any relationship attributes.	Default choice
2. Relationship Relation	Create a separate relation with PKs of both as FKs. PK is same as PK of N-side.	Only if very few tuples participate (avoid NULLs)

> **Why the N-Side?**
>
> Each entity on the N-side is related to **at most ONE** entity on the 1-side. Therefore, each tuple needs only **one FK value**.

Examples from COMPANY Database

Table 1.4 1:N Mapping Examples

Relationship	N-Side	Foreign Key Added
WORKS_FOR (Dept → Emp)	EMPLOYEE	Dno references DEPARTMENT.Dnumber
CONTROLS (Dept → Proj)	PROJECT	Dnum references DEPARTMENT.Dnumber
SUPERVISION (Recursive on Emp)	EMPLOYEE	Super_ssn references EMPLOYEE.Ssn

Step 5: Mapping of Binary M:N Relationship Types

Algorithm

1. We **cannot** use the foreign key approach for M:N because neither side can hold a single value representing the relationship.
2. **Mandatory:** Use the **Relationship Relation (Cross-Reference)** approach.
3. Create a new relation *R*. Include the PKs of the participating relations as foreign keys in *R*.
4. The **combination of these foreign keys** forms the PK of *R*.
5. Include any simple attributes of the M:N relationship in *R*.

> **Referential Triggers**
>
> Specify **CASCADE** for both ON UPDATE and ON DELETE on the foreign keys because relationship instances are **existence-dependent** on the entities they relate.

Example from COMPANY Database

Relationship: WORKS_ON (M:N between EMPLOYEE and PROJECT)

Mapped Relation: WORKS_ON(Essn, Pno, Hours)

- **FKs:** Essn references EMPLOYEE.Ssn; Pno references PROJECT.Pnumber
- **PK:** {Essn, Pno}
- **Relationship attribute:** Hours

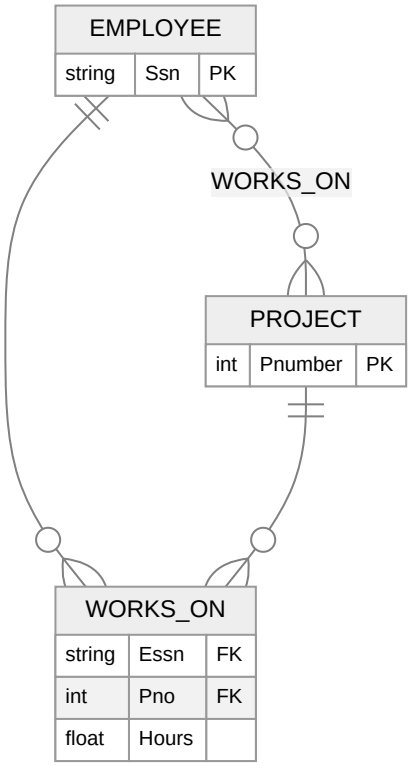

Figure 1.4: M:N Relationship Mapping (Relationship Relation)

Step 6: Mapping of Multivalued Attributes

Algorithm

1. The basic relational model doesn't allow lists/sets of values in a single tuple.
2. For each multivalued attribute A of entity E, create a new relation R.
3. Include an attribute corresponding to A, plus the primary key of the owner entity as a foreign key in R.
4. **Primary Key:** The combination of the FK and A. (If A is composite, include its simple components).

> **Modern DB Note**
>
> In newer object-relational systems allowing **array data types**, this step can be skipped by using an array attribute instead of a new table.

- **PK:** {Dnumber, Dlocation}

If Department 5 has 3 locations, there will be 3 separate tuples in this table for Dept 5.

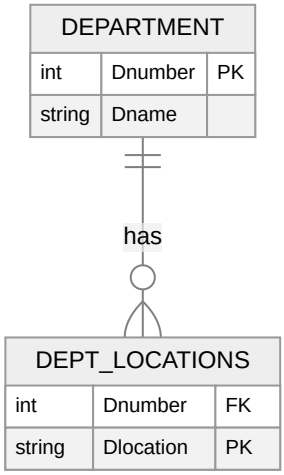

Figure 1.5: Multivalued Attribute Mapping

Step 7: Mapping of N-ary Relationship Types

Algorithm

1. Map similarly to M:N relationships using the **relationship relation** option.
2. Create a new relation *R*. Include the PKs of **all participating entity types** as foreign keys.
3. Include any simple relationship attributes.
4. **Primary Key:** Usually the combination of all the foreign keys.

Exception Constraint

If the cardinality constraint on any entity *E* is **1**, then the FK referencing *E* is **excluded from the primary key** of *R*.

Example (Ternary SUPPLY)

Relationship: Ternary SUPPLY relating SUPPLIER (*Sname*), PART (*Part_no*), and PROJECT (*Proj_name*)

Mapped Relation: SUPPLY(Sname, Proj_name, Part_no, Quantity)

- **PK:** {Sname, Proj_name, Part_no} — the combination of all three FKs

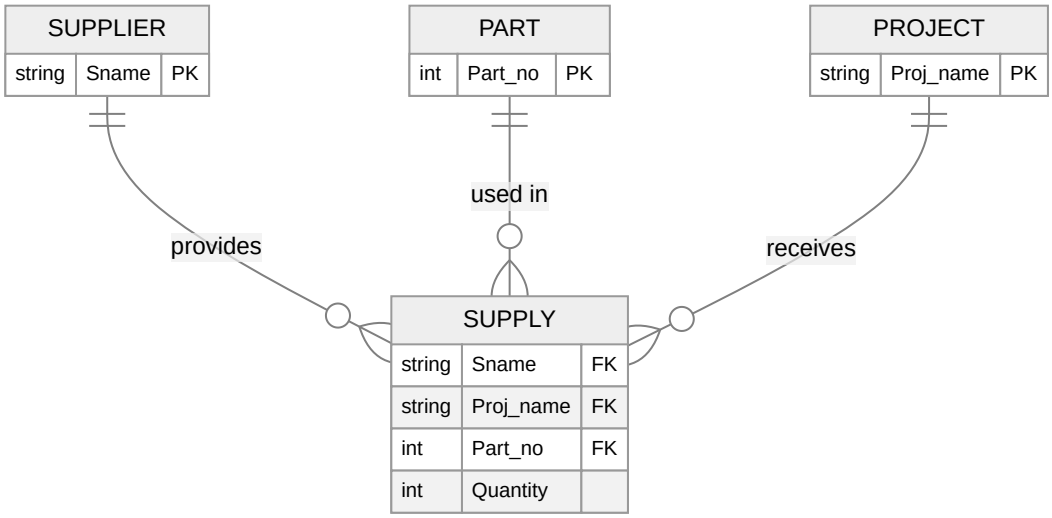

Figure 1.6: N-ary Relationship Mapping

Summary of Correspondences & Materializing Relationships via JOINs

Table 1.5 ER Model to Relational Model Correspondences

ER Model Construct	Relational Model Construct
Entity type	Entity relation
1:1 or 1:N relationship type	Foreign key (or relationship relation)
M:N relationship type	Relationship relation and two foreign keys
N-ary relationship type	Relationship relation and N foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key

Core Concept: Materializing Relationships via JOINs

In the relational model, relationships are represented logically by **sharing attributes** (PK in one table → FK in another over the same domain).

> **How to Combine Data**
>
> To materialize the relationship, use the **EQUIJOIN** operation (or **NATURAL JOIN** if attribute names match exactly) over the PK/FK pairs.

Table 1.6 JOIN Requirements by Relationship Type

Relationship Type	Mapping	Join Operations Required
1:1 or 1:N	Foreign key mapping	1 join operation
M:N	Relationship relation mapping	2 join operations
N-ary	N-ary relationship mapping	N join operations

> **Warning: Avoid Meaningless JOINs**
>
> Users must accurately know the PK/FK attributes. Applying an EQUIJOIN on non-related attributes (e.g., joining PROJECT and DEPT_LOCATIONS simply because Dlocation = Plocation) leads to **meaningless or spurious data**.

Drawbacks of the Basic Relational Model

1. **Data Repetition:** Mapping multivalued attributes requires duplicating the primary key of the owner for every value. DEPT_LOCATIONS requires three tuples for Department 5 just to list three cities.
2. **Lack of Advanced Algebra:** Basic relational algebra lacks NEST or COMPRESS operations. Cannot generate a single consolidated tuple like <'5', {'Bellaire', 'Sugarland', 'Houston'}>.
3. **Solution:** Object data models and Object-Relational systems solve this by supporting built-in list or array data types directly within attributes.

Part 2: Informal Design Guidelines for Relation Schemas

Before diving into formal normalization theory, we measure the quality of a relation schema using four core **informal guidelines**:

1. Making sure the semantics (meaning) of the attributes are clear.
2. Reducing redundant information in tuples.
3. Reducing NULL values in tuples.
4. Disallowing the possibility of generating spurious (invalid) tuples.

Guideline 1: Imparting Clear Semantics to Attributes

> **Guideline 1 Statement**
>
> Design a relation schema so that it is easy to explain its meaning. **Do not combine attributes from multiple entity types and relationship types into a single relation.**

> **Rule of Thumb**
>
> If a relation corresponds to **exactly one entity type** or **one relationship type**, it will not have semantic ambiguities.

Examples of Good Design

Table 2.1 Well-Designed Relations (Clear Semantics)

Relation	Represents	Keys
EMPLOYEE	One employee per tuple	PK: Ssn; FK: Dnumber
DEPARTMENT	One department per tuple	PK: Dnumber
PROJECT	One project per tuple	PK: Pnumber
DEPT_LOCATIONS	Multivalued attribute of department	PK: {Dnumber, Dlocation}
WORKS_ON	M:N relationship between employees and projects	PK: {Essn, Pno}

Examples of Violating Guideline 1

Table 2.2 Poorly-Designed Relations (Semantic Ambiguity)

Bad Relation	Violation
EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn)	Mixes employee attributes with department attributes
EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation)	Mixes employee attributes, project attributes, and WORKS_ON relationship attributes

> **Key Takeaway**
>
> These "fat" schemas are perfectly fine to be used as **Views** for end-users, but they are terrible as **Base Relations** (underlying stored tables) because they cause data maintenance issues.

Guideline 2: Redundant Information and Update Anomalies

A major goal of schema design is **minimizing storage space**. Grouping too many attributes together creates massive redundancy, which directly leads to **Update Anomalies**.

> **The Three Types of Update Anomalies**
>
> Using the badly designed **EMP_DEPT** relation as an example:

1. Insertion Anomalies

- **Scenario A (Consistency Risk):** To insert a new employee working in Dept 5, you must manually enter all department details (Dname, Dmgr_ssn) perfectly. If you misspell "Research", the database becomes inconsistent. Good design: just enter Dnumber = 5.
- **Scenario B (Entity Integrity Risk):** How do you insert a new department without employees? You'd need NULLs for all employee attributes. But Ssn is the PK and cannot be NULL — **violating Entity Integrity**.

2. Deletion Anomalies

- If employee 'Borg, James E.' is the last employee in "Headquarters" (Dept 1), deleting his tuple **inadvertently destroys all information about that department**. Good design keeps DEPARTMENT and EMPLOYEE as independent tables.

3. Modification Anomalies

- If Dept 5 gets a new manager, you must find and **update every single employee tuple** in Dept 5. If the system crashes halfway, or you miss a row, the database claims Dept 5 has two different managers simultaneously.

> **Guideline 2 Statement (Exam Must-Know)**
>
> Design base relation schemas so that **no insertion, deletion, or modification anomalies are present**. If anomalies must be present (e.g., for performance/materialized views), note them clearly and use automatic mechanisms (triggers or stored procedures) to ensure consistency.

Guideline 3: NULL Values in Tuples

If you create "fat" relations containing attributes that don't apply to every tuple, your database will be filled with **NULL values**.

Problems Caused by NULLs

1. **Wasted Space:** At the storage level.

2. **Unpredictable Comparisons:** SELECT and JOIN operations involving comparisons become unpredictable.
3. **Skewed Aggregations:** Functions like COUNT or SUM handle NULLs differently, leading to incorrect results.
4. **Ambiguity in Meaning:** A NULL can mean three completely different things:
   - **Not Applicable** — e.g., Visa_status for a domestic U.S. student.
   - **Unknown** — e.g., an employee's Date_of_birth is unknown.
   - **Known but Absent** — e.g., the employee has a Home_Phone_Number but it hasn't been recorded yet.

> **Guideline 3 Statement**
>
> Avoid placing attributes in a base relation whose values may frequently be NULL. If unavoidable, ensure they only apply to rare/exceptional cases.

> **Practical Example**
>
> If only 15% of employees have a private office, **DO NOT** put Office_number in EMPLOYEE. Instead, create: **EMP_OFFICES(Essn, Office_number)**. This saves 85% of tuples from carrying a meaningless NULL.

Guideline 4: Generation of Spurious Tuples

When decomposing a bad relation into smaller ones, it must be done **carefully**. If decomposed incorrectly, joining them back together generates **fake, invalid data** called spurious tuples.

Example of Terrible Decomposition

Take the bad EMP_PROJ table and split it into:

1. EMP_LOCS(Ename, Plocation)
2. EMP_PROJ1(Ssn, Pnumber, Hours, Pname, Plocation)

> **The Problem**
>
> We lost the direct link between the employee and the specific project they work on. When we try to reconstruct using NATURAL JOIN, the join condition defaults to the shared attribute: **Plocation**.

Employee "Smith, John B." works in "Bellaire" and "Sugarland". Because we join merely on Plocation, the operation creates combinations that never existed! It pairs Smith with other people's projects just because those projects happen to be in Sugarland or Bellaire.

> **Guideline 4 Statement**
>
> Design relation schemas so that they can be joined with equality conditions on attributes that are appropriately related (**Primary Key to Foreign Key pairs**). This guarantees no spurious tuples are generated. Avoid joining on matching attributes that are **not PK/FK relationships**.

> **Formal Property**
>
> The formal mathematical property that guarantees a join won't produce spurious tuples is called the **nonadditive (or lossless) join property**.

Summary of Design Guidelines

A problematic relational schema design can be identified informally by looking for:

Table 2.3 Four Warning Signs of Bad Schema Design

Sign	What It Means	Solution
Ambiguity	Mixing multiple entities/relationships making it hard to explain	Separate into entity/relationship relations
Anomalies	Redundant work during insertions/modifications; accidental data loss during deletions	Decompose to eliminate redundancy
Waste	Massive NULL values causing inefficient storage and complicated queries	Extract optional attributes to separate tables
Spurious Data	Invalid records generated during joins on non-PK/FK attributes	Ensure lossless (nonadditive) join decomposition

> **Formal Tools Coming Next**
>
> Formal tools used to prevent these issues — **Functional Dependencies** and **Normal Forms like BCNF** — are covered in Part 3.

Part 3: Functional Dependencies & Normal Forms

14.2 Functional Dependencies (FDs)

Before normalizing a database, we need a formal tool to analyze relation schemas. The single most important concept in relational schema design is the **Functional Dependency**.

Definition of Functional Dependency

> **Formal Definition**
>
> A functional dependency is a constraint between two sets of attributes from the database.
>
> - **Notation:** $X \rightarrow Y$ (Read as: "X functionally determines Y" or "Y is functionally dependent on X").
> - **Definition:** For any two tuples t_1 and t_2 in a legal relation state r of R, if they have the same value for X ($t_1[X] = t_2[X]$), they must also have the same value for Y ($t_1[Y] = t_2[Y]$).
> - **Meaning:** The values of the X component **uniquely determine** the values of the Y component.

> **Critical Note: Direction Matters**
>
> Just because $X \rightarrow Y$ holds, it does **NOT** mean $Y \rightarrow X$ holds.

Connection to Keys

If a set of attributes X is a candidate key of relation R, it means no two tuples can have the same X value. Therefore, X functionally determines **all other attributes** in R.

If X is a key, then $X \rightarrow Y$ for **all attributes** Y in R.

Semantic Property vs. Relation State

Exam Must-Know: FDs are Semantic, Not Syntactic

A functional dependency is a property of the **semantics (meaning)** of the attributes in the real world, **NOT** a property of a single specific snapshot (state) of the database.

Table 3.1 Proving vs. Disproving FDs

Action	Method	Example
Proving an FD	You **cannot** look at a populated table and automatically infer an FD. You must know the real-world rules.	{State, Driver_license_number} → Ssn holds true in the real world.
Disproving an FD	Use a **single counterexample** to definitively disprove an FD.	In TEACH, Teacher 'Smith' teaches 'Data Structures' and 'Data Management' → Teacher → Course is ruled out.

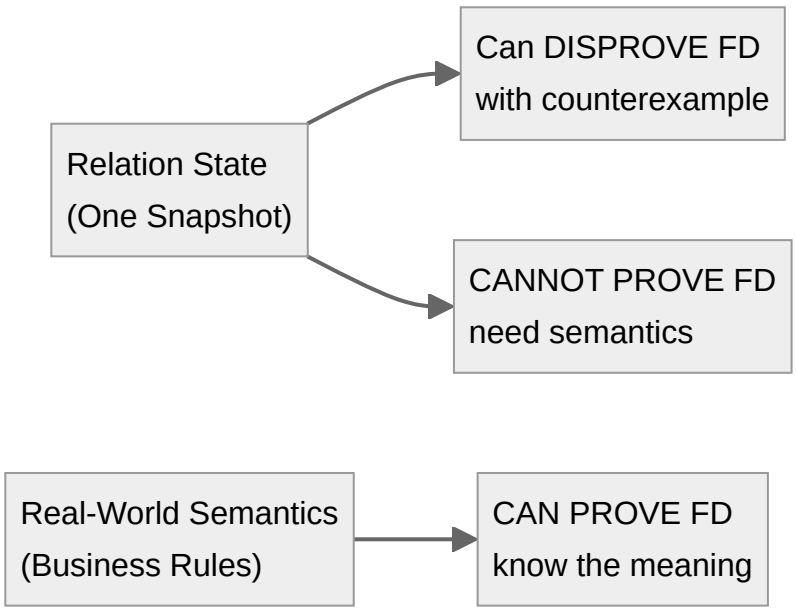

Figure 3.1: FDs depend on semantics, not data snapshots

14.3 Normal Forms Based on Primary Keys

Properties and Terminology

When decomposing relations to achieve higher normal forms, two critical properties must be maintained:

1. **Nonadditive (Lossless) Join Property:** Guarantees that no spurious (fake) tuples are generated when joining the decomposed tables back together. This is **mandatory**.
2. **Dependency Preservation Property:** Ensures every original functional dependency is represented in at least one of the new relations. This is **highly desirable**, though sometimes sacrificed.

> **Denormalization**
>
> The **intentional** process of storing joined, lower-normal-form relations to improve read/query performance (trading space/anomalies for speed).

Key Definitions Review

Table 3.2 Key Terminology for Normalization

Term	Definition
Superkey	A set of attributes that uniquely identifies a tuple.
Candidate Key	A **minimal** superkey (removing any attribute ruins uniqueness).
Primary Key (PK)	The specific candidate key chosen to identify tuples.
Prime Attribute	An attribute that is part of **any** candidate key.
Nonprime Attribute	An attribute that is **NOT** part of any candidate key.

14.3.4 First Normal Form (1NF)

> **The Rule (1NF)**
>
> The domain of an attribute must include only **atomic (simple, indivisible)** values. A tuple cannot have a set of values, nor can it contain a nested relation (a table within a table).

Violation Type 1: Multivalued Attributes

Suppose DEPARTMENT has a Dlocations attribute containing {Bellaire, Sugarland, Houston}. This violates 1NF.

Table 3.3 Solutions for 1NF Violation (Multivalued Attributes)

Solution	Approach	Quality
Solution 1 (Best)	Remove multivalued attribute, create new relation: DEPT_LOCATIONS(Dnumber, Dlocation). PK = {Dnumber, Dlocation}.	Correct, normalized
Solution 2 (Bad)	Expand PK to {Dnumber, Dlocation} and repeat all other department info on multiple rows.	Massive redundancy
Solution 3 (Bad)	Create columns Dlocation1, Dlocation2, Dlocation3.	NULLs if fewer locations; hard to query

Violation Type 2: Nested Relations

Suppose EMP_PROJ contains Ssn, Ename, and a nested table PROJS(Pnumber, Hours) inside each employee's row.

> **Solution: Unnest the Relation**
>
> Extract nested attributes into a new table and propagate the parent's PK:
>
> - New Table 1: EMP_PROJ1(Ssn, Ename)
> - New Table 2: EMP_PROJ2(Ssn, Pnumber, Hours) → PK is {Ssn, Pnumber}

> **Modern Exception**
>
> Modern BLOB/CLOB data types used for images/videos are treated as single **atomic values**, so they do not violate 1NF.

14.3.5 Second Normal Form (2NF)

> **The Rule (2NF)**
>
> A relation is in 2NF if it is in 1NF and **every nonprime attribute is fully functionally dependent on the primary key**. (There are no partial dependencies).

Full vs. Partial Dependency

- **Full Dependency:** $X \rightarrow Y$ is full if removing **any** attribute from X destroys the dependency.
- **Partial Dependency:** $X \rightarrow Y$ is partial if you can remove an attribute from X and the dependency still holds.
- **Shortcut:** If a table has a **single-attribute Primary Key**, it is automatically in 2NF (partial dependencies require a composite key).

2NF Normalization Example

Relation: EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation)

Primary Key: {Ssn, Pnumber}

Table 3.4 Dependencies in EMP_PROJ

Dependency	Type	Status
{Ssn, Pnumber} → Hours	Full Dependency	Good
Ssn → Ename	Partial Dependency	Bad! Ename depends only on part of the key
Pnumber → {Pname, Plocation}	Partial Dependency	Bad! Project details depend only on Pnumber

Decomposition to achieve 2NF:

1. EP1(Ssn, Pnumber, Hours) → Keeps the full dependency
2. EP2(Ssn, Ename) → Extracts the employee partial dependency
3. EP3(Pnumber, Pname, Plocation) → Extracts the project partial dependency

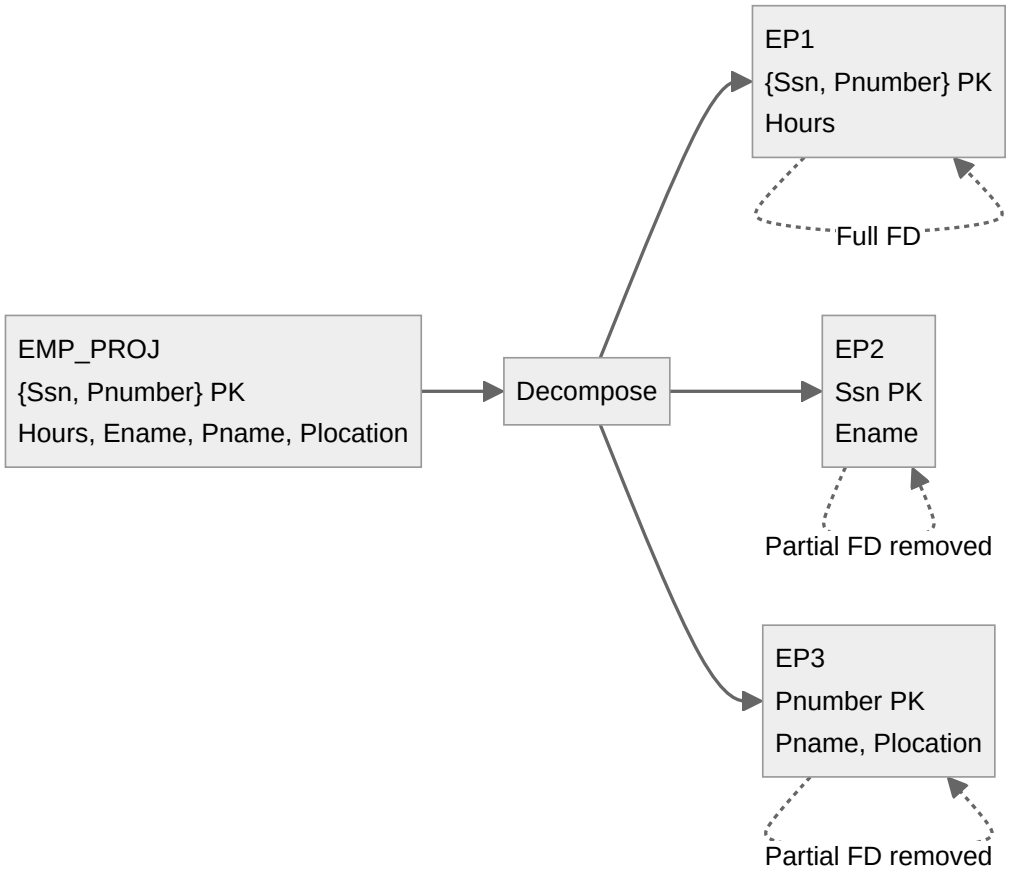

Figure 3.2: 2NF Decomposition Example

14.3.6 Third Normal Form (3NF)

> **The Rule (3NF)**
>
> A relation is in 3NF if it satisfies 2NF and **no nonprime attribute is transitively dependent on the primary key**.

Transitive Dependency

A dependency $X \rightarrow Y$ is **transitive** if there is an intermediate set of attributes Z (which is **not a candidate key**) such that $X \rightarrow Z$ and $Z \rightarrow Y$.

3NF Normalization Example

Relation: EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn)

PK: Ssn

- Dependencies: Ssn → Dnumber holds.

> **Intuitive Check**
>
> A department's name describes the **department**, not the employee. Therefore it should not be in the employee table.

Decomposition to achieve 3NF:

1. ED1(Ssn, Ename, Bdate, Address, Dnumber) → Employee info (Dnumber is a FK)
2. ED2(Dnumber, Dname, Dmgr_ssn) → Department info

14.4 General Definitions of 2NF and 3NF

The definitions in Section 14.3 only focused on the **primary key**. However, a robust relational design must evaluate **all candidate keys**.

> **General Rule for "Prime Attribute"**
>
> An attribute is prime if it is a member of **any candidate key** of the relation (not just the primary key).

14.4.1 General Definition of 2NF

> **General 2NF Rule**
>
> A relation schema R is in 2NF if every nonprime attribute A in R is **not partially dependent on any candidate key** of R.

General 2NF Example: LOTS

Relation: LOTS(Property_id#, County_name, Lot#, Area, Price, Tax_rate)

Table 3.5 Candidate Keys and FDs in LOTS

FD	Description
Property_id# → {County_name, Lot#, Area, Price, Tax_rate}	Property ID determines everything
{County_name, Lot#} → {Property_id#, Area, Price, Tax_rate}	County + Lot number also determines everything
County_name → Tax_rate	Tax rate is fixed per county
Area → Price	Price is strictly determined by area

Candidate Keys: Property_id# AND {County_name, Lot#}

The 2NF Violation: County_name → Tax_rate causes a partial dependency! Tax_rate is a nonprime attribute, and it depends on County_name, which is only **part** of the candidate key {County_name, Lot#}.

Decomposition to achieve 2NF:

1. LOTS1(Property_id#, County_name, Lot#, Area, Price) → Keeps non-violating data
2. LOTS2(County_name, Tax_rate) → Extracts the partial dependency

14.4.2 General Definition of 3NF

> **General 3NF Rule**
>
> A relation schema R is in 3NF if, whenever a nontrivial functional dependency $X \rightarrow A$ holds in R, either:
>
> a. X is a **superkey** of R, OR
>
> b. A is a **prime attribute** of R.

> **Alternative General Definition**
>
> Every nonprime attribute must be (1) **fully functionally dependent** on every key AND (2) **nontransitively dependent** on every key.

General 3NF Example: Continuing with LOTS1

Look at the newly formed LOTS1 table. It is in 2NF, but is it in 3NF?

The 3NF Violation: We still have Area → Price in LOTS1.

- Is Area (X) a superkey? **No.**
- Is Price (A) a prime attribute? **No.**
- Therefore, it violates the general definition of 3NF (Price is transitively dependent on the keys via Area).

Decomposition to achieve 3NF:

1. LOTS1A(Property_id#, County_name, Lot#, Area)
2. LOTS1B(Area, Price)

> **Efficiency Note**
>
> If a relation passes the general 3NF test, it automatically passes 2NF as well, meaning you could decompose LOTS straight into LOTS1A, LOTS1B, and LOTS2 without stopping at 2NF.

14.5 Boyce-Codd Normal Form (BCNF)

BCNF was proposed as a **simpler, but stricter**, form of 3NF. Look at clause (b) in the general 3NF definition: "or A is a prime attribute". This clause creates a **loophole** that allows certain problematic functional dependencies to "slip through" because the right-hand side happens to be part of a candidate key. **BCNF closes this loophole.**

> **The Rule (BCNF)**
>
> A relation schema R is in BCNF if whenever a nontrivial functional dependency $X \rightarrow A$ holds in R, X **is a superkey of** R.
>
> Notice that clause 'b' from 3NF has been **completely removed**.

BCNF vs 3NF Example: LOTS1A

Relation: LOTS1A(Property_id#, County_name, Lot#, Area)

Candidate Keys: Property_id# and {County_name, Lot#}

New Constraint: Suppose lot sizes are strictly assigned by county (e.g., DeKalb only has 0.5-1.0 acre lots, Fulton has 1.1-2.0). This gives us: **Area → County_name**

Testing LOTS1A against Normal Forms

Table 3.6 LOTS1A Normal Form Analysis

Normal Form	Test	Result
Is it in 3NF?	Area → County_name: Area is NOT a superkey, BUT County_name IS a prime attribute. Satisfies clause (b).	**YES** (technically in 3NF)
Is it in BCNF?	Area → County_name: Area is NOT a superkey. No other clauses.	**NO** (violates BCNF)

> **The Problem with 3NF Here**
>
> Even though LOTS1A is technically in 3NF, it still causes **redundancy**. If 1,000 lots are 0.6 acres, we will repeatedly store that they belong to DeKalb county.

Decomposition to achieve BCNF:

1. LOTS1AX(Property_id#, Area, Lot#)
2. LOTS1AY(Area, County_name)

The Dependency Preservation Trade-off

> **Critical Exam Concept: BCNF vs Dependency Preservation**
>
> By decomposing into BCNF, notice what happened to FD2: {County_name, Lot#} → Area. This functional dependency is **lost**. We can no longer easily enforce it because County_name and Lot# no longer exist in the same table.
>
> **This proves:** While BCNF is excellent for eliminating redundancy, it sometimes forces us to sacrifice the **Dependency Preservation Property**.

Chapter 15: Inference Rules and Closures for FDs

When dealing with functional dependencies, it's often necessary to deduce new dependencies from a known set. We use formal rules and algorithms to do this systematically.

> **Notation Simplification**
>
> Moving forward, sets of attributes will be concatenated for convenience. Example: $X \cup Y$ will be written simply as XY.

Armstrong's Axioms (The Basic Inference Rules)

Armstrong proposed three foundational rules (IR1 - IR3) that are both **sound** and **complete**:

- **Sound:** Any FD you derive is guaranteed to be true in every legal relation state.
- **Complete:** Using these rules repeatedly will generate the entire possible set of derived FDs (called the **Closure**, denoted F^+).

Table 3.7 Armstrong's Axioms

Rule	Name	Statement	Meaning
IR1	Reflexive	If $Y \subseteq X$, then $X \rightarrow Y$	A set always determines itself or any subset (trivial FDs)
IR2	Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$	Adding same attributes to both sides preserves the FD
IR3	Transitive	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	FDs behave transitively

Additional Derived Inference Rules

The following rules can be mathematically proven using IR1, IR2, IR3:

Table 3.8 Derived Inference Rules

Rule	Name	Statement	Meaning
IR4	Decomposition	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$	Split up the right-hand side
IR5	Union	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$	Combine dependencies with same left-hand side
IR6	Pseudotransitive	If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$	Substitute left-hand side with another set that determines it

Critical Cautionary Notes

- $X \to YZ$ does **NOT** imply $X \to Y$ and $X \to Z$ separately **without the decomposition rule**.
- $X \to Y$ and $X \to Z$ does **NOT** imply $Y \to Z$ or $Z \to Y$.

Determining the Closure of a Set of Attributes (X^+)

Database designers usually only specify the semantically obvious FDs (set F). To find everything else that depends on a specific set of attributes X, we calculate its **Closure** (denoted X^+).

Definition

The closure X^+ is the **complete set of attributes** that are functionally determined by X, based on the given set of dependencies F.

Algorithm 15.1: Calculating X^+

Algorithm

Input: A set F of FDs on relation schema R, and a subset of attributes X.

1. Set $X^+ := X$ (Start by assuming X determines itself via the Reflexive Rule).
2. Repeat:
   - $oldX^+ := X^+$
   - For every functional dependency $Y \to Z$ in F:
   - If all attributes of Y are currently inside X^+:
   - Add the attributes of Z to X^+ ($X^+ := X^+ \cup Z$)
3. Until X^+ stops changing (i.e., $X^+ == oldX^+$)

Example of Finding Closures

Relation: CLASS(Classid, Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity)

Given FDs (F):

- FD1: Classid → Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity
- FD2: Course# → Credit_hrs
- FD3: Course#, Instr_name → Text, Classroom
- FD4: Text → Publisher
- FD5: Classroom → Capacity

Table 3.9 Closure Calculation Examples

Closure	Steps	Result	Conclusion
{Classid}$^+$	Starts as {Classid}; applying FD1 determines everything.	{Classid, Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity}	**Candidate Key!** (closure = entire schema)
{Course#}$^+$	Starts as {Course#}; applying FD2 gives Credit_hrs.	{Course#, Credit_hrs}	NOT a candidate key (missing Instr_name, etc.)
{Course#, Instr_name}$^+$	Apply FD2 → Credit_hrs; FD3 → Text, Classroom; FD4 → Publisher; FD5 → Capacity	{Course#, Instr_name, Credit_hrs, Text, Classroom, Publisher, Capacity}	**NOT a candidate key** (Classid is missing!) — proves a course taught by a specific instructor can have multiple class sections.

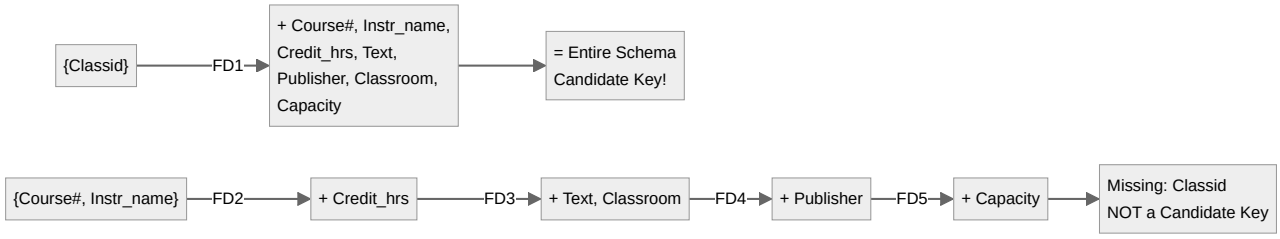

Figure 3.3: Closure Calculation Flow

Part 4: Exam Practice Q&A

This section contains practice questions with detailed answers to help you prepare for your exam.

ER-to-Relational Mapping Questions

Q1: When mapping a regular (strong) entity type to a relation, should you include foreign keys in Step 1?

> **A:** No. Step 1 only maps the entity's own simple attributes. Foreign keys and relationship attributes are added in later steps (Steps 3-7 depending on the relationship type). This keeps entity relations clean and semantically clear.

Q2: A weak entity DEPENDENT has a partial key Dependent_name and owner EMPLOYEE with PK Ssn. What is the primary key of the DEPENDENT relation?

> **A:** The PK is {Essn, Dependent_name}. Essn is the FK referencing EMPLOYEE.Ssn, and it is combined with the partial key Dependent_name to uniquely identify each dependent. You must also specify CASCADE for ON UPDATE and ON DELETE because a weak entity has existence dependency on its owner.

Q3: For a 1:1 relationship between EMPLOYEE and DEPARTMENT where every department must have a manager but not every employee manages a department, where should the foreign key go?

> **A:** The FK (Mgr_ssn) should go into the DEPARTMENT table because DEPARTMENT has **total participation** (every department has a manager). Putting it in EMPLOYEE would create massive NULLs since most employees don't manage a department.

Q4: Why can't we use the foreign key approach for M:N relationships?

> **A:** In an M:N relationship, one entity instance on either side can be related to **multiple** instances on the other side. A single FK column can only hold one value per tuple, so it cannot represent multiple relationships. Therefore, we **must** create a separate relationship relation (cross-reference table) with the PKs of both entities as FKs, and their combination becomes the PK.

Q5: How many join operations are needed to materialize an M:N relationship compared to a 1:N relationship?

> **A:** A 1:N relationship mapped with foreign keys requires **1 join operation** (join the two tables on the FK). An M:N relationship requires **2 join operations** (first join one entity table with the relationship relation, then join the result with the other entity table).

Design Guidelines Questions

Q6: What are the four informal design guidelines for relation schemas?

> **A:** (1) **Clear semantics** — each relation should represent one entity or relationship type; (2) **Reduce redundancy** — avoid update anomalies; (3) **Reduce NULL values** — don't store frequently-null attributes in base relations; (4) **Avoid spurious tuples** — decompose so joins on PK/FK pairs produce only valid data.

Q7: Explain the three types of update anomalies with examples.

> **A:Insertion anomalies:** Can't insert a department without employees (would need NULL PK) or must duplicate department info for every new employee. **Deletion anomalies:** Deleting the last employee in a department accidentally deletes the department itself. **Modification anomalies:** Changing a department manager requires updating every employee in that department; partial updates cause inconsistency.

Q8: Why are "fat" relations like EMP_DEPT acceptable as Views but not as Base Relations?

> **A:** As Views, they provide convenient read-only access to joined data without storing redundant data. As Base Relations, they actually store duplicated data, causing insertion, deletion, and modification anomalies. Views are virtual; Base Relations are physical storage.

Q9: What does the nonadditive (lossless) join property guarantee?

> **A:** It guarantees that when you decompose a relation and then join the decomposed tables back together using equality conditions on PK/FK pairs, you get **exactly** the original tuples — no spurious (fake) tuples and no lost tuples.

Functional Dependencies & Normal Forms Questions

Q10: Can you determine a functional dependency just by looking at the current data in a table?

> **A:** No. You can only **disprove** an FD with a counterexample from data. To **prove** an FD, you must understand the real-world semantics and business rules. A current snapshot might coincidentally satisfy an FD that doesn't actually hold in general.

Q11: What is the difference between a partial dependency and a transitive dependency?

> **A:** A **partial dependency** occurs when a nonprime attribute depends on only **part** of a composite primary key (violates 2NF). A **transitive dependency** occurs when a nonprime attribute depends on another nonprime attribute, which in turn depends on the primary key (violates 3NF). Partial = depends on part of key; Transitive = depends on something that depends on key.

Q12: A relation has a single-attribute Primary Key. What normal forms is it automatically in?

> **A:** It is automatically in **2NF** (assuming it's also in 1NF). Partial dependencies require a composite key to exist, so with a single-attribute PK, partial dependencies are impossible. However, it may still violate 3NF if there are transitive dependencies.

Q13: What is the difference between 3NF and BCNF?

> **A:** In 3NF, for every nontrivial FD $X \to A$, either X is a superkey OR A is a prime attribute. In BCNF, the second condition is removed — X **must be a superkey**, period. BCNF is stricter. Every BCNF relation is in 3NF, but not vice versa. BCNF may require sacrificing dependency preservation.

Q14: Why might we intentionally choose 3NF over BCNF?

> **A:** Because achieving BCNF sometimes requires decomposing in a way that **loses functional dependencies** (dependency preservation is sacrificed). If enforcing a particular FD across multiple tables is critical for data integrity, designers may accept 3NF to keep that dependency enforceable within a single table.

Q15: State Armstrong's three axioms and what "sound" and "complete" mean.

> **A:IR1 Reflexive:** If $Y \subseteq X$, then $X \to Y$. **IR2 Augmentation:** If $X \to Y$, then $XZ \to YZ$. **IR3 Transitive:** If $X \to Y$ and $Y \to Z$, then $X \to Z$. **Sound** means any derived FD is guaranteed true. **Complete** means the axioms can derive every FD that logically follows from F.

Q16: How do you use closure (X^+) to find candidate keys?

> **A:** Calculate X^+ for a set of attributes X. If X^+ contains **all attributes** of the relation schema, then X is a superkey. If no proper subset of X also determines all attributes, then X is a **candidate key** (minimal superkey).

Q17: Given FDs: A → BC, B → D, D → E. Find A⁺.

> **A:** Start with $A^+ = \{A\}$. Apply A → BC: $A^+ = \{A, B, C\}$. Apply B → D: $A^+ = \{A, B, C, D\}$. Apply D → E: $A^+ = \{A, B, C, D, E\}$. If these are all attributes, A is a candidate key.

Q18: A relation R(A, B, C, D) has FDs: AB → C, B → D, and candidate keys AB and BC. Is R in 3NF? Is it in BCNF?

> **A:** Check B → D: B is not a superkey (B^+ = {B, D} ≠ all attributes). D is not a prime attribute (D is not part of any candidate key: AB or BC). So B → D violates 3NF. Therefore, R is **NOT in 3NF** and consequently **NOT in BCNF**. Decompose into R1(B, D) with PK B, and R2(A, B, C) with PK AB (or BC).

Q19: Decompose EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn) into 3NF.

> **A:** PK is Ssn. FDs: Ssn → {Ename, Bdate, Address, Dnumber}; Dnumber → {Dname, Dmgr_ssn}. The violation: Dname and Dmgr_ssn are transitively dependent on Ssn through Dnumber. Decompose: (1) EMPLOYEE(Ssn, Ename, Bdate, Address, Dnumber) where Dnumber is FK; (2) DEPARTMENT(Dnumber, Dname, Dmgr_ssn).

Q20: What are the two critical properties that must be maintained when decomposing relations?

> **A:** (1) **Nonadditive (Lossless) Join Property** — mandatory; guarantees no spurious tuples when rejoining. (2) **Dependency Preservation Property** — highly desirable; ensures every original FD can be checked within at least one decomposed table without needing joins.

Part 5: Quick Reference Cheat Sheet

Normal Forms at a Glance

Table 5.1 Normal Forms Summary

Normal Form	Rule	What It Prevents	How to Achieve
1NF	All attributes are atomic (indivisible)	Multivalued attributes, nested relations	Split into separate relations or use atomic domains
2NF	1NF + no partial dependencies of nonprime attributes on any candidate key	Partial dependencies (nonprime attribute depends on part of a composite key)	Decompose to separate the partial dependency
3NF	2NF + no transitive dependencies of nonprime attributes on any candidate key; OR for every FD $X \to A$: X is superkey OR A is prime	Transitive dependencies (nonprime depends on nonprime)	Decompose to remove the transitive chain
BCNF	For every nontrivial FD $X \to A$: X is a superkey	All remaining anomalies from prime-attribute loophole in 3NF	Decompose; may lose dependency preservation

ER-to-Relational Mapping Cheat Sheet

Table 5.2 Seven-Step Mapping Quick Reference

Step	ER Construct	Relational Result	Key Notes
1	Regular Entity	Entity relation with simple attributes; choose PK	No FKs yet
2	Weak Entity	Relation with owner's PK as FK + partial key	CASCADE triggers; PK = owner PK + partial key
3	Binary 1:1	FK in entity with total participation (or merge if both total)	Avoid NULLs; include relationship attributes
4	Binary 1:N	FK on the N-side	Add relationship attributes to N-side
5	Binary M:N	New relationship relation with both PKs as FKs	PK = composite of both FKs; CASCADE triggers
6	Multivalued Attribute	New relation with owner PK as FK	PK = FK + multivalued attribute
7	N-ary Relationship	New relation with all participant PKs as FKs	If any entity has cardinality 1, exclude its FK from PK

Armstrong's Axioms Quick Reference

Table 5.3 Inference Rules Summary

Rule	Statement	Use
Reflexive (IR1)	$Y \subseteq X \Rightarrow X \to Y$	Trivial FDs; starting point for closures
Augmentation (IR2)	$X \to Y \Rightarrow XZ \to YZ$	Expanding both sides equally
Transitive (IR3)	$X \to Y, Y \to Z \Rightarrow X \to Z$	Chaining dependencies
Decomposition (IR4)	$X \to YZ \Rightarrow X \to Y, X \to Z$	Splitting RHS
Union (IR5)	$X \to Y, X \to Z \Rightarrow X \to YZ$	Combining RHS
Pseudotransitive (IR6)	$X \to Y, WY \to Z \Rightarrow WX \to Z$	Substituting LHS

Closure Algorithm Summary

Algorithm: Finding X^+

1. Initialize: $X^+ = X$
2. Repeat until no change:
   - For each FD $Y \to Z$ in F:
   - If $Y \subseteq X^+$ then $X^+ = X^+ \cup Z$
3. If X^+ = all attributes → X is a superkey
4. If no proper subset of X also determines all → X is a candidate key

Key Formulas & Definitions

Table 5.4 Essential Formulas

Concept	Formula / Definition
Full Dependency	$X \to Y$ is full if removing any attr from X destroys it
Partial Dependency	$X \to Y$ is partial if some proper subset $X' \subset X$ also determines Y
Transitive Dependency	$X \to Z \to Y$ where Z is not a candidate key and Y is nonprime
Lossless Join Test	$R_1 \bowtie R_2$ is lossless if $R_1 \cap R_2$ is a superkey of R_1 or R_2
Dependency Preservation	Each FD in F must be enforceable in at least one decomposed relation

Final Exam Checklist

- Know all 7 mapping steps in order
- Know the 4 informal guidelines and their violations
- Be able to identify insertion, deletion, and modification anomalies
- Understand why FDs are semantic, not based on data snapshots
- Know the definitions of 1NF, 2NF, 3NF, and BCNF precisely
- Be able to decompose relations to achieve each normal form
- Know Armstrong's 6 inference rules
- Be able to compute attribute closures and find candidate keys
- Understand the lossless join vs. dependency preservation trade-off